I/O Systems

Dept. of Computer Science

Hanyang University

I/O Hardware

- Variety of I/O devices
- Common concepts
  - Port (ex. Serial port)
  - Bus (daisy chain or shared direct access)
  - Controller (simple chip or separate circuit board called host adapter)
- I/O instructions control devices
  - Controller has one or more registers for data and control signals
- Processor give commands and data to a controller via
  - Special I/O instructions to an I/O port address
  - Memory-mapped I/O
    - Device control registers are mapped into the address space of the process
    - CPU executes I/O requests using the standard data-transfer instructions

I/O Hardware

- I/O port consists of four registers
  - Status register
    - Contains bits that can be read by the host
    - Busy bit, error bit
  - Control register
    - Can be written by the host to start a command or to change the mode of a device
  - Data-in register
    - Read by the host to get input
  - Data-out register
    - Written by the host to send output

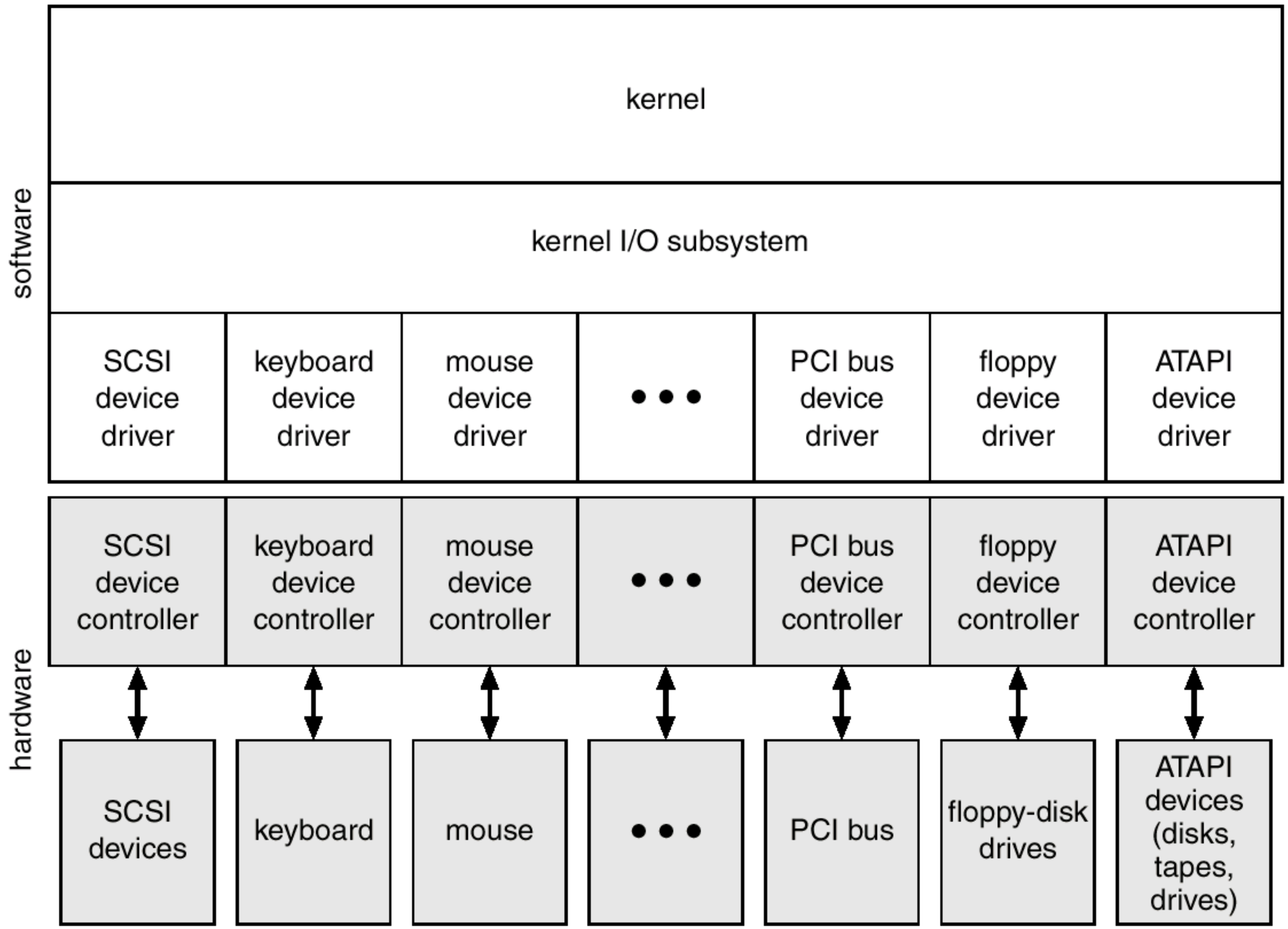
Polling

- Determines state of device
  - `Busy bit` in the status register
- Busy-wait cycle to wait for I/O from device
  - If the controller and device are fast, polling is a reasonable one
- Polling becomes inefficient when it is attempted repeatedly, yet rarely finds a device to be ready for service, while other useful CPU processing remains undone
- In such instances, It is better to arrange for the hardware controller to notify the CPU when the device becomes ready for service: *interrupt*

Interrupts

- CPU hardware has a wire called the interrupt-request line
  - Triggered by I/O device
  - CPU senses after executing every instruction
- Detecting a signal, CPU saves state and jumps to the interrupt handler routine at a fixed address in memory
- Interrupt handler
  - Determines the cause of the interrupt, performs the necessary processing, and executes a *return from interrupt* instruction to return the CPU to the prior state
- Two interrupt-request lines
  - Nonmaskable: reserved for events such as unrecoverable memory errors
  - Maskable: used by device controllers and can be turned off by CPU before critical instruction sequences
- Interrupt vector to dispatch interrupt to correct handler
- Interrupt mechanism also used for exceptions
  - Divide by zero, accessing a protected memory address, etc

A Kernel I/O Structure



Application I/O Interface

- I/O system calls encapsulate device behaviors in generic classes to make I/O devices treated in a standard, uniform way
- Device-driver layer hides differences among I/O controllers from kernel
- Devices vary in many dimensions
  - Character-stream or block
  - Sequential or random-access
  - Sharable or dedicated
  - Speed of operation
  - read-write, read only, or write only

Block and Character Devices

- Block devices include disk drives
  - Commands include `read`, `write`, `seek`
  - Raw I/O (ex. Database system) or file-system access
  - Memory-mapped file access possible
    - Data transfers are handled by the same mechanism as that used for demand-paged virtual-memory access
- Character devices include keyboards, mice, serial ports
  - Commands include `get`, `put`
  - Convenient for input devices which produce data for input spontaneously
  - Good for output devices such as printers or audio boards, which naturally fit the concept of a linear stream of bytes

Blocking and Nonblocking I/O

- Blocking I/O
  - Process suspended until I/O completed
  - Application is moved from run queue to wait queue
  - Easy to use and understand
- Non-blocking I/O
  - I/O call returns as much as available
  - Returns quickly with a return value that indicates how many bytes were transferred (read or written)
- Asynchronous I/O
  - Process runs while I/O executes
  - Requests a transfer that will be performed in its entirety, but that will complete at some future
    - Non-blocking I/O returns immediately with whatever data are available: the full number of bytes requested, fewer, or none at all
  - Difficult to use
  - I/O subsystem signals process when I/O completed

Kernel I/O Subsystem

- Scheduling
  - I/O request ordering via per-device queue
  - try to be fair
- Buffering
  - Store data in memory while transferring between devices
  - To cope with device speed mismatch
  - To cope with device transfer size mismatch
  - To maintain "*copy semantics*"
    - "The version of the data written to disk is guaranteed to be the version of at the time of the application system call, independent of any subsequent changes in the application's buffer"
    - Copy the application data into a kernel buffer before returning control to the application. Disk write is performed from the kernel buffer, not the application buffer.

Kernel I/O Subsystem

- Caching
  - Key to performance
  - A region of fast memory that holds copies of data
  - Always just a *copy* of an item that resides elsewhere
    - Buffer holds the only copy of a data item

- Spooling
  - hold output for a device
  - If device can serve only one request at a time
    - Eg., Printing

Kernel I/O Subsystem

- Error Handling
  - OS can recover from disk read failure, device unavailable, transient write failures so that a complete system failure is not the usual result of each minor problem
  - Return an error number or code when I/O request fails
  - System error logs hold problem reports
- Kernel Data Structures
  - Kernel keeps state info for I/O components, including open file tables, network connections, character device state
  - Many, many complex data structures to track buffers, memory allocation, “dirty” blocks

Performance

- I/O is a major factor in system performance
  - Places heavy demands on CPU to execute device driver, kernel I/O code
  - Context switches due to interrupts
  - Data copying
  - Network traffic especially stressful
- I/O Performance Improvement
  - Reduce number of context switches
  - Reduce data copying
  - Reduce interrupts by using large transfers or polling
  - Use DMA
  - Balance CPU, memory, bus, and I/O performance for highest throughput

Concluding the OS course…

Operating System is…

a collection of Cheating Schemes.